Learn Python

Teacher Edition

Minute-by-minute session playbooks · transition scripts · predicted misconceptions · Socratic prompts

A 10-session, ~20-hour (two hours per session) accelerated Python course for a PhD-in-Education learner with no prior coding experience.

Generated 2026-06-27 · Instructional content original

Learn Python — TEACHER Edition

Everything in the student syllabus, **plus** the instructor scaffolding: a minute-by-minute clock for each two-hour session, transition scripts (how to move between blocks without dead air), predicted misconceptions for *this* learner, Socratic prompts (DeepTutor-style — ask, don't tell), and an explicit **"if you're behind, cut this"** line per session.

How to read this document

Each session has:

- **Pre-flight** — what to have open/ready before the student arrives.
- **The clock** — a 120-minute breakdown (with a mid-session break). Keep a timer visible. The numbers are targets, not law.
- **Transitions** — short scripted lines to hand off cleanly between blocks.
- **Predicted misconceptions** — where THIS learner (expert in research, novice in code) will stumble.
- **Socratic prompts** — questions to ask instead of explaining; let them derive it.
- **Cut line** — the first thing to drop if you're running over.

Universal pacing principles (this learner is fast)

- **Talk less than you want to.** He reads fast and abstracts well. Default to "here's the rule, here's the trap, now you try."
- **Protect — and fill — the practice block.** The ~60-minute hands-on block (split around the break) is where learning happens. It's packed on purpose: this learner finishes individual tasks quickly, so the block carries *more* tasks rather than more minutes. If concept overruns, steal from your own talking, never from his typing.
- **Predict-then-run is the engine, especially S2.** Always have him *commit to an answer out loud* before running. The cognitive surprise is the teaching moment.
- **Don't pad.** Each clock is tight by design — no long buffers to fill with lecture. If he's ahead, give him the next practice task or a stretch goal, not more talking.
- **Carry a running "misconceptions log"** (DeepTutor "learning memory"): note every trap he hit this session; re-surface it as a 60-second warm-up next session.

SESSION 1 — Running Python, Variables & Types

Pre-flight: terminal + VS Code open; `examples/session-01/` ready; a deliberately broken line staged to show a traceback.

The clock (120 min)

- **0:00–0:08 — Orientation.** Why Python, why this re-ordered path, how the session works. Don't oversell; he wants to start.
- **0:08–0:32 — Concept.** REPL vs script; `python file.py`. Variables as *labels on objects* (use Connection Map #1). The five core types. `input()` → always `str`. f-strings. `int()/float()/str()`.
- **0:32–0:48 — Live code.** Build `greet.py` then a tiny `interest.py` ("years to graduation") together; deliberately trigger and *read* one traceback.
- **0:48–1:16 — Practice (packed).** Student works through `examples/session-01/practice.md` (BMI/GPA-converter style script + the extra short tasks). You stay quiet; answer only when asked. Hand him the next task the moment he finishes one.
- **1:16–1:24 — Break.**
- **1:24–1:54 — Practice, continued.** Push into the harder/extra tasks and one self-chosen variation; he should leave having typed a complete small program unaided.
- **1:54–1:58 — Traps recap.** The `input()`-returns-string trap; `print(a, b)` vs `print(a + b)`; integer vs float division preview.
- **1:58–2:00 — Summary + 3-question quiz.**

Transitions

- Concept→Live: *"Enough theory — watch me make these mistakes so you don't have to."*
- Live→Practice: *"Your turn. Break it on purpose at least once and read the error out loud."*
- Practice→Traps: *"Before we close — here are the two things that get everyone in week one."*

Predicted misconceptions (this learner)

- Expects `input()` to give a number (it's a string) → `"5" + "3" == "53"`.
- Thinks `=` asserts equality (math habit). Reinforce label-on-object framing now; it pays off in S2/S5.
- Reads tracebacks top-down and panics. Teach: **read the last line first.**

Socratic prompts

- "What type do you think `input()` hands back? How could we check?" (→ `type(...)`)
- "Why did `'5' + '3'` give `'53'`? What would make it `8`?"

Cut line: drop the compound-interest live demo; keep `greet.py` + the traceback.

SESSION 2 — The Dynamic-Typing Traps

Pre-flight: Open `cheatsheets/traps-and-gotchas.md` and `examples/session-02/traps_demo.py`. This is the most important session — protect it. Tell him so.

The clock (120 min) — this session is *predict-then-run* almost end to end, so the hands-on practice is woven through every block, not saved for one slot.

- **0:00–0:06 — Warm-up.** Re-surface S1 traps (60-sec quiz from your misconceptions log).
- **0:06–0:34 — Block A: `==` vs `is`.** Value vs identity (Connection Map #3: same GPA vs same person). Show `is` with lists, `None` (`is None`), and the small-int cache wrinkle (label it "implementation detail — never rely on it").
- **0:34–1:02 — Block B: the number traps.** `bool` ⊂ `int` (`True == 1`, `5 + True`, `sum([True,False,True])` = dummy coding, Connection Map #4); `3 == 3.0`; **float precision** `0.1 + 0.2` (Connection Map #5) → `math.isclose`, round-for-display.
- **1:02–1:10 — Break.**
- **1:10–1:40 — Block C: cross-type comparison + type checking.** `5 == "5"` is `False` but `5 > "5"` raises `TypeError`; sequence comparison element-by-element (`[1,2]==(1,2)` is `False`); `isinstance(x,(int,float))` vs `type(x) is int`; truthiness of `0/""/[]/None`.
- **1:40–1:56 — Practice.** `examples/session-02/practice.md`: a "predict the output" gauntlet, then a `clean_score()` that handles int/float/str-number inputs safely.
- **1:56–2:00 — Summary.** Hand him the trap cheat sheet as his permanent reference; quiz.

Transitions

- A→B: *"Identity vs value — hold that. Now the same question for numbers, where Python is sneakier."*
- B→C: *"Mixing numbers is fine. Mixing a number with text? Watch — sometimes False, sometimes an explosion."*
- C→Practice: *"Cover the right column of the cheat sheet. Predict each line, then run."*

Predicted misconceptions (this learner)

- Will assume `is` is just a stylistic `==` (very common). Nail it with mutable-list demo.
- Will trust `==` on floats out of stats habit ("I round anyway") — connect to measurement error but stress the *cause* is binary storage, not data.
- May over-generalize the `TypeError` and think `5 == "5"` also errors. It returns `False` — show it.
- Will reach for `type(x) == int`; redirect to `isinstance` and explain inheritance (sets up `bool`⊂`int`).

Socratic prompts

- "Two students, same 3.7 GPA. `==`? `is`? Why?"
- "You already sum 0/1 dummies. So what's `sum([True, False, True])`? Why?"

- "If `0.1 + 0.2` isn't `0.3`, is the bug in the data or in the computer? How would you test 'close enough'?"
- "`5 == '5'` is False, fine. So why does `5 > '5'` *crash*? What does the computer not know how to do?"

Cut line: drop the small-int cache wrinkle and the walrus aside; never cut Blocks A–C core or the practice.

SESSION 3 — Control Flow: Conditionals & Loops

Pre-flight: `examples/session-03/`; a messy nested-`if` snippet staged for the refactor demo; `Ctrl+C` ready to demo killing an infinite loop. (This session merges what many courses split across two — a fast learner clears decisions and repetition together, since loops are just decisions that repeat.)

The clock (120 min)

- **0:00–0:06 — Warm-up** (S2 traps recap).
- **0:06–0:30 — Concept.** *Conditionals:* `if/elif/else`; comparison ops; **chained comparisons** (`90 <= x < 100` — they love this, it reads like math); `and/or/not`; short-circuit; `and/or` return an *operand*, not a bool. *Loops:* `while` (+ infinite-loop demo, `Ctrl+C`); `for ... in`; `range` (off-by-one!); `break` / `continue`; the `while True: ... break` validation pattern; **`enumerate`** and **`zip`** as the antidote to `range(len(...))`.
- **0:30–0:46 — Live code.** A Likert→label classifier using a chained comparison and an early `return`; then sum/average a list of scores two ways (index vs `enumerate` / `zip`); finish with a robust "ask until valid" prompt using `while True`.
- **0:46–1:14 — Practice (packed).** `examples/session-03/practice.md`: grade-band classifier (test every boundary), boolean-logic predictions, average + pass/fail with `zip`, a real validation loop, and the mutate-while-iterating trap fix. Keep handing him the next task.
- **1:14–1:22 — Break.**
- **1:22–1:52 — Practice, continued.** Tougher boundaries, a second validation loop, and the mutate-while-iterating fix done two ways (copy vs comprehension).
- **1:52–1:58 — Traps recap.** `if x == True` (just `if x`); `is None` not `== None`; `=` vs `==` typo; `range(1,5)` excludes 5; **mutating a list while looping it**; reaching for `range(len(...))` instead of `enumerate` / `zip`.
- **1:58–2:00 — Summary + quiz.**

Transitions

- Concept→Live: *"Watch me turn an ugly five-level `if` into three readable lines — then loop over a whole roster."*
- Live→Practice: *"Build the grade classifier; make it pass all the band boundaries — the edges are where bugs hide. Then loop the roster, and if you catch yourself writing `range(len(...))`, stop and use `enumerate`."*

Predicted misconceptions

- Will write `if score >= 90 and score < 100` and not know about chaining → show `90 <= score < 100`.
- Will write `if passed == True`. Ask "what type is `passed` already?"
- Boundary errors (`>=` vs `>`) at grade cutoffs — make him test 89.999 / 90 / 90.001.
- Off-by-one with `range`. Have him print `list(range(1,5))` to see it.
- `range(len(x))` index habit (from R/SPSS vectorized thinking) — push `enumerate` / `zip`.

- Trying to remove items from a list *while iterating it* → demo the bug, then the fix (iterate a copy / build a new list).

Socratic prompts

- "`x = 5 and 0` — what's `x`? Why isn't it `True` / `False`?"
- "How would a mathematician write `between 90 and 100`? Python lets you write it that way."
- "You need both the position and the value. What's cleaner than indexing?"
- "Two parallel lists, names and scores. How do you walk them together?"

Cut line: drop `match/case` and `for/else` (mention they exist, point to the cheat sheet); keep chained comparisons, `enumerate` / `zip`, and the validation loop.

SESSION 4 — Data Structures: list, tuple, dict, set

Pre-flight: `examples/session-04/`; a "list of dicts = tidy dataset" diagram (Connection Map #6).

The clock (120 min)

- **0:00–0:06 — Warm-up.**
- **0:06–0:30 — Concept.** `list` (mutable) vs `tuple` (immutable) vs `dict` (key→value) vs `set` (unique). Indexing & **slicing**. Nesting → **list of dicts = a dataset**. Comprehensions (list + dict). `sorted(key=lambda ...)`.
- **0:30–0:46 — Live code.** Build a roster as a list of dicts; sort by score; dedupe survey answers with a `set`; rewrite a loop as a comprehension.
- **0:46–1:14 — Practice (packed).** Group students by grade band into a dict; build `{name: average}` with a dict comprehension; slice/sort tasks (`examples/session-04/practice.md`). Feed the next task as soon as one lands.
- **1:14–1:22 — Break.**
- **1:22–1:52 — Practice, continued.** Nest a list of dicts deeper, sort by two keys, and rewrite one more loop as a comprehension; show the aliasing bug live before they hit it.
- **1:52–1:58 — Traps recap. Aliasing** (`b = a` shares the list) vs `a.copy()`; `[[0]*3]*3` shared rows; shallow vs deep copy; sequence comparison recap (ties back to S2); **mutable default arg** preview (full treatment S5).
- **1:58–2:00 — Summary + quiz.**

Transitions

- Concept→Live: *"A list of dicts is just a tidy dataset — rows are dicts, keys are your variables."*
- Live→Practice: *"Group the roster by grade band. Then make a `{name: average}` in one line."*

Predicted misconceptions

- **Aliasing** is the big one — he'll expect `b = a` to copy. Show `a.append(...)` changing `b`. Tie to S1 "label on object."
- Confusing tuple immutability with "you can't have a tuple of lists" — you can; the *tuple* is fixed, contents may not be.
- Comprehension syntax order (`[expr for x in xs if cond]`) — have him read it as "expr, for each x, when cond."

Socratic prompts

- "`b = a; a.append(99)` — what's in `b`? Why? (Remember: labels, not boxes.)"
- "Survey gave duplicate free-text answers. What structure removes duplicates for free?"

Cut line: drop deep-copy/`[[0]*3]*3`; keep aliasing basics + comprehensions.

SESSION 5 — Functions, Scope & Reusability

Pre-flight: `examples/session-05/` ; the **mutable-default-arg** demo staged (this is the headline).

The clock (120 min)

- **0:00–0:06 — Warm-up.**
- **0:06–0:30 — Concept.** `def` , parameters (positional/keyword/default), `return` vs `print` , `*args` / `**kwargs` , scope (LEGB), `global` (and why to avoid it), docstrings, **type hints** (+ "not enforced; `mypy` checks them").
- **0:30–0:46 — Live code.** Refactor S3/S4 inline code into `class_average(scores)` , `letter_grade(score)` ; then the **mutable-default bug** live: `def add(x, bag=[])` accumulating across calls → fix with `bag=None` .
- **0:46–1:14 — Practice (packed).** Write a small library of grade functions with type hints + docstrings, plus a `*args` aggregator and a keyword-default formatter (`examples/session-05/practice.md`).
- **1:14–1:22 — Break.**
- **1:22–1:52 — Practice, continued.** Re-implement S3/S4 logic as clean functions, add docstrings + type hints to every one, and reproduce then fix the mutable-default bug unaided.
- **1:52–1:58 — Traps recap.** Mutable default; late-binding closures; forgetting `return` (function returns `None`); `UnboundLocalError` from assigning a global.
- **1:58–2:00 — Summary + quiz.**

Transitions

- Concept→Live: *"This next bug has burned every Python programmer at least once. Watch."*
- Live→Practice: *"Build your grade-functions module — these are the pieces of your capstone."*

Predicted misconceptions

- Will not believe the default list persists between calls until shown twice. Run it, then explain *when* defaults are evaluated (once, at definition).
- Thinks a function that `print` s has "returned" the value → show `x = show(...)` is `None` .
- Will expect type hints to enforce types at runtime. Show `add("a","b")` still runs.

Socratic prompts

- "I called `add(1)` three times and the list keeps growing. When do you think `bag=[]` actually runs?"
- " `print` vs `return` — which one lets the *next* function use the result?"

Cut line: drop late-binding closures; never cut the mutable-default demo.

SESSION 6 — Recursion & Recursive Thinking

Pre-flight: `examples/session-06/`; a nested-JSON-shaped dict staged for the `deep_sum` demo; have `sys.getrecursionlimit()` ready and the `runaway` overflow demo queued.

The clock (120 min)

- **0:00–0:06 — Warm-up.** Re-surface an S5 trap from your misconceptions log.
- **0:06–0:30 — Concept.** The two parts: a **base case** and a **recursive case** that moves toward it. Trace `factorial(3)` on the board as a stack that builds, then unwinds. Recursion vs iteration (factorial both ways). Name the cost: each pending call is a stack frame; Python has **no tail-call optimization** (`sys.getrecursionlimit()` ≈ 1000).
- **0:30–0:46 — Live code.** `countdown` / `factorial`; then `deep_sum` over a nested dict — the payoff, since a single loop can't reach the bottom of arbitrarily nested data; then trigger a `RecursionError` with `runaway` and read it together.
- **0:46–1:14 — Practice (packed).** `examples/session-06/practice.md`: recursive sum, string reverse (recursion vs loop), `flatten`, `depth`, and the two trap-fixes. Hand out the next task as each lands.
- **1:14–1:22 — Break.**
- **1:22–1:52 — Practice, continued.** Harder nested-data recursion (`flatten` / `depth` variants), convert one recursion to a loop and back, and trace a stack on paper before running it.
- **1:52–1:58 — Traps recap.** Unreachable base case → stack overflow; forgetting to `return` the recursive call → silent `None`; recursion isn't free; when a loop simply reads clearer.
- **1:58–2:00 — Summary + quiz; point ahead to exceptions (S7).**

Transitions

- Concept→Live: *"Watch the stack build up and then collapse — that's the whole trick."*
- Live→Practice: *"Your turn. Say the base case out loud before you write the function — that's where the bugs hide."*

Predicted misconceptions (this learner)

- Will write the recursive case but forget to `return` it → silent `None`. Show it once.
- Will fear infinite recursion everywhere; reassure — a *reachable* base case is the guarantee.
- From a vectorized stats background, may not see when recursion beats a loop → the nested-data demo is the "aha."
- May assume recursion is a free, elegant swap for a loop → show the `RecursionError` and the ~1000 limit.

Socratic prompts

- "What's the smallest input where the answer is obvious without recursing? That's your base case."
- "Your data is a list that can contain lists. What kind of function matches a thing defined in terms of itself?"
- "`factorial(2000)` by recursion vs by loop — which one risks crashing, and why?"

Cut line: drop the `depth`/string-reverse practice tasks; keep base/recursive case, `deep_sum` on nested data, and the `RecursionError` demo.

SESSION 7 — Exceptions & Defensive Code

Pre-flight: `examples/session-07/`; a CSV-ish list with dirty values ("N/A", "", "7" on a 1–5 scale).

The clock (120 min)

- **0:00–0:06 — Warm-up.**
- **0:06–0:30 — Concept.** Errors vs exceptions; `try/except/else/finally`; common types (`ValueError`, `KeyError`, `ZeroDivisionError`, `FileNotFoundError`); `raise ValueError(...)`; **EAFP** ("easier to ask forgiveness") vs LBYL; `assert`.
- **0:30–0:46 — Live code.** Harden `get_int()` / `clean_likert()` to survive blanks, "N/A", out-of-range; show a first `pytest` test (`test_clean.py`) including `pytest.raises`.
- **0:46–1:14 — Practice (packed).** Validate a list of raw survey responses, collecting good values and a report of bad ones; add a `raise` for impossible input and one `pytest.raises` test (`examples/session-07/practice.md`).
- **1:14–1:22 — Break.**
- **1:22–1:52 — Practice, continued.** Add a custom exception, harden against two more dirty-value cases, and write a second `pytest.raises` test; compare an LBYL vs EAFP version of the same check.
- **1:52–1:58 — Traps recap. Bare `except:`** (catches everything, even `Ctrl+C`); catching too broad; silently swallowing errors; using exceptions for normal control flow excessively.
- **1:58–2:00 — Summary + quiz.**

Transitions

- Concept→Live: *"Your real survey data WILL have 'N/A' in a numeric column. Let's make code that shrugs it off."*
- Live→Practice: *"Clean this dirty response list; keep a log of every value you rejected and why."*

Predicted misconceptions

- Will reach for `if/else` checks (LBYL) everywhere; introduce EAFP as the Pythonic default but be honest both are valid.
- Will write `except:` bare — show why `except ValueError:` is safer.
- Confuses `raise` (throw) with `return`; and `assert` (a developer check, can be disabled) with input validation (must always run).

Socratic prompts

- "User typed 'seven' instead of 7. Catch it *before* (`if`) or *after* (`try`)? Trade-offs?"
- "Why is a bare `except:` dangerous? What might you accidentally swallow?"

Cut line: drop the `pytest` test → just use `assert` statements; keep `try/except` validation.

SESSION 8 — Files, Libraries & Research Data

Pre-flight: ship a sample `students.csv` and `survey.csv` in `examples/session-08/`; confirm `pip` works; have `pandas` install ready (or pre-installed) for the teaser.

The clock (120 min)

- **0:00–0:06 — Warm-up.**
- **0:06–0:30 — Concept.** `open` / `with` (context manager: auto-close); file modes (`r/w/a`; `w` **overwrites!**); reading lines; **CSV** via `csv.DictReader` / `DictWriter` (rows as dicts → ties to S4); `json` briefly; `import`; `pip install`; researcher stdlib: `statistics`, `random`, `datetime`, `pathlib`.
- **0:30–0:46 — Live code.** Read `students.csv` into a list of dicts, compute the class mean with `statistics.mean`, write a summary CSV. Then a short `pandas` **teaser**: same task in 3 lines (`read_csv`, `.describe()`), framed as "here's your next course."
- **0:46–1:14 — Practice (packed).** Read `survey.csv`, compute per-item means, write `survey_summary.csv`; add a `datetime`-stamped filename and a `pathlib` existence check (`examples/session-08/practice.md`).
- **1:14–1:22 — Break.**
- **1:22–1:52 — Practice, continued.** Mean score by major, a second summary file, and a round-trip (write then re-read your own CSV); finish with the optional `pandas` three-liner on the same data.
- **1:52–1:58 — Traps recap.** `"w"` silently destroys data; forgetting `newline=""` with `csv` (blank rows on Windows); forgetting `\n`; encoding (`utf-8`); reading a file twice (cursor at end).
- **1:58–2:00 — Summary + quiz.**

Transitions

- Concept→Live: *"This is the session your actual research data shows up. Let's read a real CSV."*
- Teaser framing: *"Everything you just did by hand, pandas does in three lines — that's your next course, not today's. But now you know what it's doing underneath."*

Predicted misconceptions

- Will open with `"w"` to read and wonder why the file is empty. Stress mode meanings.
- Expects to iterate a file object twice without re-opening; show the exhausted-cursor effect.
- May jump to pandas and skip the fundamentals — hold the line: hand-rolled first, pandas as dessert.

Socratic prompts

- "Why does `with` matter even if your program crashes? What does it guarantee?"
- "You read the file once and the second loop is empty. Where did the data 'go'?"

Cut line: drop the `pandas` teaser and `json`; keep `with`, modes, and `csv.DictReader/Writer`.

SESSION 9 — Regular Expressions & Text Cleaning

Pre-flight: `examples/session-09/`; a list of messy free-text responses, emails, and "Last, First" names staged for live demos.

The clock (120 min)

- **0:00–0:06 — Warm-up.** Re-surface an S8 trap from your misconceptions log.
- **0:06–0:30 — Concept.** Why regex for a researcher (validate/extract/clean/qualitative-coding, Connection Map #9). **Raw strings** `r"..."`. The survival tokens (`. \d \w \s + * ? {m,n} ^ $ [] () |`). Stress `.` matches *any* char (use `\.`).
- **0:30–0:46 — Live code.** The four functions: `re.search` / `fullmatch` / `findall` / `sub`. Validate an email (`fullmatch`), extract dept+number with capture groups, collapse whitespace with `sub`.
- **0:46–1:14 — Practice (packed).** `examples/session-09/practice.md`: email validator, extract codes, count `#hashtags` across responses, flip `"Last, First"`, and one case where `.split()` beats regex.
- **1:14–1:22 — Break.**
- **1:22–1:52 — Practice, continued.** Build patterns on real messy text (capture groups, the hashtag count, the name flip) at regex101 first, then in code; decide once where a string method wins.
- **1:52–1:58 — Traps recap.** `.` matches anything; forgot `r"..."`; `re.search` returns `None`; regex vs string methods.
- **1:58–2:00 — Summary + quiz.**

Transitions

- Concept→Live: *"Four functions cover almost everything: search, fullmatch, findall, sub. Watch."*
- Live→Practice: *"Your turn — and every pattern is a raw string, no exceptions."*

Predicted misconceptions

- Forgets raw strings → backslash chaos. Always `r"..."`.
- Assumes `.` matches only a dot → show it matches any char; `\.` for literal.
- Calls `.group()` on a `None` result → teach the `if m:` guard.
- Over-uses regex where `.split()` / `.strip()` / `.replace()` are clearer.

Socratic prompts

- "You want every response mentioning a theme. Is that a *form* match (regex) or a *meaning* match (human coding)? What can regex actually catch?"
- "`5 > "5"` crashed weeks ago. Why does `re.search(...).group()` crash when there's no match?"

Cut line: drop the `findall`/Counter hashtag-mining demo; keep validate + extract + `sub`.

SESSION 10 — Modules, OOP & the Pythonic Toolkit

Pre-flight: `examples/session-10/` (`grades.py` staged for the import demo, `Student` class ready); the generator-exhaustion demo queued.

The clock (120 min)

- **0:00–0:06 — Warm-up.** Re-surface an S9 trap.
- **0:06–0:22 — Modules.** Move grade functions into `grades.py`, `import` them; the `if __name__ == "__main__":` guard (file as both script and library).
- **0:22–0:46 — OOP.** A small `Student` class: `__init__`, `self` ("this particular student"), a method, `__str__`, a validating `@property` setter (Connection Map #10), then brief inheritance with `super()`.
- **0:46–1:14 — Practice (packed).** Build the validating `Student`, add `GradStudent(super())`, then one comprehension + `map` + `filter` + a generator (`examples/session-10/practice.md`).
- **1:14–1:22 — Break.**
- **1:22–1:52 — Practice, continued + Pythonic toolkit.** Tour comprehensions, `map`/`filter`, `enumerate`/`zip`, generators/`yield`, the walrus `:=` by handing out tasks rather than lecturing; have him name one case where a dict beats a class.
- **1:52–1:58 — Traps recap.** `self` confusion; a generator exhausts after one pass; over-using a class where a function/dict fits; forgetting the `__main__` guard.
- **1:58–2:00 — Summary + quiz; course wrap, point to the capstone.**

Transitions

- Modules→OOP: *"Functions in a file is reuse. Now let's bundle data* and *behavior — a class."*
- OOP→Practice: *"Build your `Student`, then we'll tour the moves that make code read like the experts'."*

Predicted misconceptions

- `self` looks redundant/magical — it's just "this particular instance."
- Treats a generator like a list (iterates once) → show the second pass is empty.
- Thinks the `__main__` guard is boilerplate → show the demo block firing on run but not on import.
- Reaches for a class when a function or dict would do — name when OOP earns its keep.

Socratic prompts

- "Your operational definition of 'student' — what attributes and rules belong to it? That's your class."
- "A million rows won't fit in memory. What does `yield` give you that a list doesn't?"

Cut line: compress the Pythonic tour to comprehensions + `enumerate`/`zip`; defer `map`/`filter`/walrus to the cheat sheet. Keep modules + the validating class.

SESSION 11 (Optional) — Capstone

Role shift: you stop teaching and start *coaching*. He drives; you ask questions and unblock.

- **0:00–0:15 — Brief & plan.** He restates the goal and sketches the steps aloud (pseudocode). You only check the plan is sound.
- **0:15–1:10 — Build.** He codes the Gradebook & Survey Analyzer (`assessments/capstone-project.md`). Intervene only when stuck >3 min; prefer a question over an answer.
- **1:10–1:18 — Break.**
- **1:18–1:45 — Build, continued.** Finish the report CSV, then tackle one stretch goal (a `Student` class, a regex validation, or the recursive nested-data total).
- **1:45–1:55 — Review.** Walk his code for the traps from S2/S4/S5 (identity, aliasing, mutable defaults). Praise readability.
- **1:55–2:00 — Debrief & next steps.** Point to pandas/visualization as the genuine next course.

Coaching prompts: "What's your data structure?" · "What happens if that cell is blank?" · "Is that comparing value or identity?" · "Could that be one comprehension?"

Instructor's running checklist (use across all sessions)

- [ ] Timer visible; practice block protected *and* packed.
- [ ] Misconceptions log updated after each session; warm-up next session pulls from it.
- [ ] Every trap demoed via **predict-then-run**, not narration.
- [ ] Each new concept hooked to the **Connection Map** before syntax.
- [ ] Student typed everything himself; you talked less than half the session.
- [ ] End-of-session quiz given; capstone kept in view as the destination.

Appendix A — Master Course Outline

Master Course Outline — Learn Python

> Single source of truth. The student and teacher syllabi both derive from this. **10 core two-hour sessions** + 1 optional capstone session (~20 hours core, ~22 with the capstone).

How the topics are sequenced

A typical intro-Python course teaches roughly: functions/variables → conditionals → loops → exceptions → libraries → unit tests → file I/O → regular expressions → OOP → assorted "power-tools."

We **re-sequence** for a fast adult learner: dynamic-typing fundamentals come forward into Session 2; conditionals and loops are taught together as one "control flow" session (they're the two halves of the same idea, and a fast learner clears them quickly); and the power-tools (comprehensions, generators, `*args`, type hints) are folded into the sessions where they naturally belong instead of left to the end. **Recursion** gets its own session right after Functions (S6): it's a function that calls itself, so it lands best while function mechanics are fresh — and Data Structures (S4) is already in hand for the nested-data payoff.

Design rules (from the e-learning pipeline)

- Every session = two hours, structured **Concept → Live Example → Practice → (break) → more Practice → Traps → Summary**.
- **Practice is the biggest block (~60 min, split around a mid-session break) and is packed** — this learner moves fast, so each session's practice carries enough tasks to fill the time without padding.
- Every session has 2–4 learning objectives; every objective is testable in that session's quiz.
- Every abstract idea ships with a runnable, education-flavored example.
- Difficulty rises monotonically; nothing is used before it's introduced (except clearly-flagged teasers).

Session 1 — Running Python, Variables & Types

Objectives

1. Run Python both interactively (REPL) and as a `.py` script; read a traceback without panic.
2. Use variables and the core built-in types: `int`, `float`, `str`, `bool`, `None`.
3. Do I/O with `input()` / `print()`, format with f-strings, and convert types with `int()` / `float()` / `str()`.

Key idea the beginner misses: `input()` *always* returns a `str`; numbers from users must be converted.

Session 2 — The Dynamic-Typing Traps (THE CORE SESSION)

Objectives

1. Distinguish **value equality (`==`)** from **identity (`is`)** and know when each is correct.
2. Predict results when mixing numeric types: `bool` ⊂ `int`, `int` / `float`, and **float precision**.
3. Check types correctly with `isinstance()` vs `type()`, and reason about truthiness.

This is the session the whole course is built around — it front-loads every trap in the learner's brief (`==` vs `is`, `True == 1`, `3 == 3.0`, `0.1 + 0.2`, `5 == "5"` vs `5 > "5"`, sequence comparison, `isinstance`). Everything later assumes this fluency.

Session 3 — Control Flow: Conditionals & Loops

Objectives

1. Write `if` / `elif` / `else` with comparison & logical operators and **chained comparisons**; use `and` / `or` / `not` correctly (short-circuit, operand-return) and avoid `if x == True`.
2. Write `for` and `while` loops; control them with `break` / `continue`; mind `range` off-by-one.
3. Iterate the Pythonic way with `enumerate` and `zip`, and build the `while True:` validation loop.

Trap focus: `if x == True` / `is None`, `=` vs `==`, `range(1,5)` excludes 5, mutating a list while iterating it, reaching for `range(len(...))` instead of `enumerate` / `zip`.

Session 4 — Data Structures: list, tuple, dict, set

Objectives

1. Choose between `list`, `tuple`, `dict`, and `set`; index, slice, and nest them.
2. Sort with `sorted(..., key=...)` and lambdas; build **list/dict comprehensions**.
3. Reason about **mutability & aliasing** (copy vs reference, shallow vs deep copy).

Trap focus: aliasing (`b = a`), `list * n` shared rows, mutable default arguments (preview), element-by-element sequence comparison, set deduplication of survey data.

Session 5 — Functions, Scope & Reusability

Objectives

1. Define functions with positional, keyword, default, `*args`, and `**kwargs` parameters.
2. Explain scope (LEGB), `return` vs `print`, and avoid the `UnboundLocalError` / `global` trap.
3. Document with docstrings and annotate with **type hints** (and know hints aren't enforced).

Trap focus: the **mutable default argument** bug, late-binding closures, implicit `None` return.

Session 6 — Recursion & Recursive Thinking

Objectives

1. Write a recursive function with a correct **base case** and a **recursive case**; trace the **call stack** and convert between recursion and iteration.
2. Reason about recursion's cost: a missing base case → `RecursionError`, and Python has no tail-call optimization (each pending call keeps a stack frame, default limit ~1000).
3. Apply recursion to **naturally nested data** (nested lists/dicts, JSON, trees) where a single loop is awkward.

Trap focus: missing/unreachable base case (infinite recursion → stack overflow), forgetting to `return` the recursive call (silent `None`), assuming recursion is free, reaching for recursion where a plain loop reads more clearly.

Session 7 — Exceptions & Defensive Code

Objectives

1. Handle errors with `try` / `except` / `else` / `finally`; raise `ValueError` etc. deliberately.
2. Validate real, messy human/research input robustly (EAFP "ask forgiveness" style).
3. Use `assert` and write a first `pytest` test for a function.

Trap focus: bare `except:`, catching too broadly, swallowing errors silently.

Session 8 — Files, Libraries & Research Data

Objectives

1. Read/write text with `open` / `with`; understand file modes and why `with` matters.
2. Load and write **CSV** survey/gradebook data with `csv.DictReader` / `DictWriter`; touch `json`.
3. `import` standard-library tools a researcher reaches for (`statistics`, `random`, `datetime`, `pathlib`) and install a third-party package with `pip` (guided `pandas` teaser).

Trap focus: `"w"` silently overwrites, forgotten `newline=""` / `\n`, file encoding.

Session 9 — Regular Expressions & Text Cleaning

Objectives

1. Write patterns with raw strings and the core tokens (`. \d \w \s + * ? {m,n} ^ $ [] () |`).
2. Use `re.search` / `fullmatch` / `findall` / `sub` to validate, extract (capture groups), and clean text.
3. Apply regex to real research text: validate IDs/emails, extract codes, normalize and mine free responses.

Trap focus: `.` matches *any* char (use `\.`), forgetting raw strings, `re.search` returns `None` , reaching for regex where a string method is clearer.

Session 10 — Modules, OOP & the Pythonic Toolkit

Objectives

1. Split code into modules and `import` them; understand the `if __name__ == "__main__":` guard.
2. Model a domain entity with a small **class** (`__init__`, `self`, `__str__`, a validating `@property`, brief inheritance with `super()`).
3. Apply the "Pythonic" toolkit: comprehensions, `map` / `filter`, `enumerate` / `zip`, generators/ `yield`, the walrus `:=`.

Trap focus: `self` confusion, a generator exhausts after one pass, over-using a class where a function/dict fits.

Session 11 (Optional) — Capstone Project *(integrative)*

Objective: Independently build one small, end-to-end program on a real-ish education dataset. Default brief: **"Gradebook & Survey Analyzer"** — read a CSV of students + Likert responses, clean and validate it, compute summary statistics, flag at-risk students, and write a report CSV. Alternative briefs are listed in `assessments/capstone-project.md`.

Coverage check (every core topic lands somewhere)

Topic	Where it lives here
Functions & variables	S1, S5
Conditionals	S3 (+ traps in S2)
Loops	S3 (+ iterating data in S4)
Recursion	S6 (nested data ties to S4)
Exceptions	S7
Libraries	S8
Unit tests	S7 (+ modules in S10)
File I/O	S8 (+ data structures S4)
Regular expressions	S9
Modules & OOP	S10
Power-tools (sets, comprehensions, `*args`, type hints, generators, `map` / `filter`)	S4, S5, S10

Scaling to the time budget

- **~16 hours:** fold the Pythonic-toolkit half of S10 into S4/S5 and trim the `pandas` teaser (recursion, S6, stays — it's a core skill, not an add-on).
- **~20 hours:** run S1–S10 as written, two hours each (recommended).
- **~22 hours:** add the S11 capstone.

Appendix B — Connection Map (education-research bridges)

Connection Map — Python ⇄ Education-Research Background

> From the *personalized-syllabus* method: each new programming concept is bridged to something the student already knows from education research, and — crucially — **where the bridge breaks** is named, so the new concept isn't quietly collapsed into the familiar one. Use these as the "hooks" when introducing each topic; they cut learning time dramatically for an expert adult.

---

1. Variables & assignment *(S1)*

- **Maps onto:** named quantities in a stats model — `n`, `mean`, `alpha`.
- **Where it breaks:** in math, `x = x + 1` is a contradiction; in Python `=` is an *action* (rebind the name), not an assertion of equality. A variable is a **label stuck on an object**, not a box that holds a value. This single reframing prevents most aliasing confusion later.

2. Types & dynamic typing *(S1, S2)*

- **Maps onto:** measurement levels — nominal/ordinal/interval/ratio. Python's `str` / `bool` / `int` / `float` are loosely analogous (categorical vs counted vs continuous).
- **Where it breaks:** SPSS/R columns have *one declared type per column*; a Python variable can hold a string now and an int later. The discipline a column gives you for free, you must supply yourself. This is exactly why the comparison traps in S2 exist.

3. `==` vs `is` *(S2)*

- **Maps onto:** the difference between **two questionnaires with identical answers** (equal in value) and **the same physical respondent** (identical individual). Two students can have the same GPA (`==`) without being the same person (`is`).
- **Where it breaks:** the analogy is exact for objects, but Python adds an implementation wrinkle (small-integer caching) that has no analogue in research — flagged as a "do not rely on this."

4. Boolean as a subclass of int (`True == 1`) *(S2)*

- **Maps onto: dummy coding** — you already encode "treatment = 1, control = 0" and then *sum* the dummies to count cases. Python literally does this: `sum([True, False, True]) == 2`.

- **Where it breaks:** in your data the 1/0 is a convention you imposed; in Python it's baked into the language, so `True + True == 2` works even when you didn't intend arithmetic on a flag.

5. Float precision (`0.1 + 0.2 != 0.3`) *(S2)*

- **Maps onto: rounding/measurement error**. You already never test two measured scores for exact equality; you use tolerances and report to 2 decimals.
- **Where it breaks:** here the error isn't in the data — it's in how the *computer* stores decimals in binary. Same instinct (`math.isclose`, round for display), new cause.

6. Lists / dicts / DataFrames *(S4, S8)*

- **Maps onto:** a `list` of `dict`s is a **tidy dataset**: each dict is a *row/respondent*, each key is a *variable/column*. A `dict` alone is one record's variable→value map.
- **Where it breaks:** a spreadsheet enforces rectangularity; a list of dicts does not — rows can have missing or extra keys, which is the source of many bugs (and why we validate in S7).

7. Functions *(S5)*

- **Maps onto:** a **statistical formula or a coding scheme**: defined once, applied to many cases, same rule every time → reproducibility, the thing you care about most as a researcher.
- **Where it breaks:** functions can carry *hidden state* (mutable defaults, globals) so the "same input → same output" promise can silently fail. That trap (S5) is the whole reason to learn scope.

8. Exceptions & validation *(S7)*

- **Maps onto: data cleaning** — out-of-range Likert values, blank cells, "N/A" typed into a numeric field. You already have a mental model of dirty data; exceptions are how code reacts to it.
- **Where it breaks:** cleaning in SPSS is a one-time batch pass; in a program you decide *at runtime*, per value, whether to skip, fix, or stop — a more granular control than a recode syntax file.

9. Regular expressions *(S9)*

- **Maps onto: search-and-filter in a corpus / qualitative coding** — finding every response matching a pattern, extracting IDs, normalizing free-text.
- **Where it breaks:** regex matches *surface form*, not meaning. It's powerful for structure (emails, dates, IDs) and treacherous for semantics — the opposite trade-off from human coding.

10. Classes / OOP *(S10)*

- **Maps onto:** an **operational definition / construct** — a `Student` class bundles the attributes and behaviors you've decided "count" as a student, like an operationalized variable.

- **Where it breaks:** a construct is a measurement choice; a class is also *behavior* (methods) and *enforced rules* (validation in setters), so it's a construct that can defend its own integrity.

11. Recursion *(S6)*

- **Maps onto:** a **hierarchy / nested structure** — coding schemes with sub-codes, threaded discussion data, folder trees of data files, nested JSON survey exports. A procedure "defined in terms of itself" mirrors data that contains smaller copies of itself.
- **Where it breaks:** recursion is elegant only when the structure is genuinely nested and the base case is reachable; for a flat sequence, a loop is clearer, and Python's stack limit (~1000, no tail-call optimization) makes very deep recursion a liability rather than a virtue.

Questions to hold while learning (Socratic spine)

These recur across sessions; the teacher should keep returning to them.

1. *Is this a question about value, or about identity?* (drives `==` vs `is`, copy vs alias)
2. *What type is this right now, and who decided that?* (dynamic typing discipline)
3. *Could this input be dirty? What happens if it is?* (defensive mindset)
4. *Is the same rule guaranteed to run the same way every time?* (reproducibility / pure functions)
5. *Is there a more Pythonic, more readable way to say this?* (the Zen of Python's "readability counts")

Appendix C — Per-Session Quizzes & Answer Keys

Per-Session Quizzes (with answer keys)

Each quiz is 3–5 questions, ~5 minutes, given at the end of the session. Every question maps to a session objective. **Teacher:** ask the student to *predict* code output before they run it — the surprise is the assessment. Answers are at the end of each session block.

Session 1 — Variables & Types

1. What type does `input("x: ")` return, always?
2. What does `"5" + "3"` evaluate to? How do you get `8`?
3. What is `int(3.9)`? What is `round(3.9)`?
4. (Practical) Write one line that prints a float `score` to 2 decimal places.

Answers: 1. `str`. 2. `"53"`; use `int("5") + int("3")`. 3. `3` (truncates) and `4`.

4. `print(f"{score:.2f}")`.

Session 2 — Dynamic-Typing Traps

1. `a = [1,2]; b = [1,2]` — is `a == b`? is `a is b`? Why?
2. What is `True + True`? Why?
3. Is `0.1 + 0.2 == 0.3` True or False? How should you compare them?
4. What does `5 == "5"` give? What about `5 > "5"`?
5. Is `[1,2] == (1,2)` True or False? Why?

Answers: 1. `==` True (same value), `is` False (different objects). 2. `2` (bool is a subclass of int). 3. False; use `math.isclose(0.1+0.2, 0.3)` (or round). 4. `False`; `5 > "5"` raises `TypeError` (can't order int vs str). 5. False — a list and a tuple are different types.

Session 3 — Control Flow: Conditionals & Loops

1. Rewrite `if x >= 90 and x < 100:` using a chained comparison.
2. What is `5 and 0`? What is `0 or "hi"`? Why aren't they `True` / `False`?
3. Why is `if passed == True:` not ideal? Write the better version.
4. What does `list(range(1, 5))` produce?
5. Why can removing items from a list *while looping it* go wrong, and what's a clean fix?

Answers: 1. `if 90 <= x < 100:`. 2. `0` and `"hi"` — `and` / `or` return an operand, not a bool. 3. `passed` is already a bool; write `if passed:`. 4. `[1, 2, 3, 4]` (5 excluded).

5. Removing shifts indices so the loop skips elements; iterate a copy or build a new list (`[x for x in xs if keep(x)]`).

Session 4 — Data Structures

1. Which are mutable: list, tuple, dict, set?
2. What does `xs.sort()` return? How do you get a *new* sorted list?
3. `a = [1,2]; b = a; a.append(3)` — what is `b`? Why?
4. (Practical) One-line dict comprehension mapping each name in `names` to `0`.

Answers: 1. list, dict, set are mutable; tuple is not. 2. `None` (sorts in place); `sorted(xs)`. 3. `[1, 2, 3]` — `b` is an alias for the same list. 4. `{n: 0 for n in names}`.

Session 5 — Functions & Scope

1. What's the difference between `return` and `print`?
2. Why is `def f(x, items=[])` dangerous? What's the fix?
3. What does a function return if it has no `return` statement?
4. Are type hints enforced at runtime?

Answers: 1. `return` hands a value to the caller; `print` only displays. 2. The default list is created once and persists across calls; use `items=None` then create inside. 3. `None`.

4. No — they're documentation; `mypy` checks them optionally.

Session 6 — Recursion & Recursive Thinking

1. What two parts must every recursive function have?
2. What error comes from a base case that's never reached, and why doesn't Python just keep going?
3. What does this `fact` return for `fact(4)`, and why? `python def fact(n): if n <= 1: return 1 n * fact(n - 1)`
4. Name one situation where a plain loop is the better choice than recursion.
5. Why is recursion a natural fit for nested data (a list of lists, or nested JSON)?

Answers: 1. A **base case** (stops) and a **recursive case** (calls itself on a smaller input, moving toward the base case). 2. `RecursionError: maximum recursion depth exceeded` — Python has no tail-call optimization, so each pending call keeps a stack frame until it hits the limit (~1000). 3. `None` — the recursive case computes `n * fact(n-1)` but never `return`s it, so the function falls off the end. 4. A flat sequence, or work deep enough to exceed the recursion limit (a loop has no frame cost). 5. The data is *defined in terms of itself* (a list may contain lists), so a function defined in terms of itself mirrors its shape and can reach every level.

Session 7 — Exceptions

1. Which exception does `int("N/A")` raise?
2. Why is a bare `except:` dangerous?
3. When should you use `raise` vs `assert`?
4. (Practical) Wrap `int(value)` so it returns `None` on failure.

Answers: 1. `ValueError`. 2. It catches everything (even Ctrl+C and your own typos) and can hide bugs. 3. `raise` to validate real/untrusted input; `assert` for developer sanity checks (can be disabled). 4. `try: return int(value) except (ValueError, TypeError): return None`.

Session 8 — Files & Data

1. What does opening a file in `"w"` mode do to existing contents?
2. Why prefer `with open(...)` over `open()` / `close()`?
3. After `csv.DictReader`, what type is each row?
4. CSV values read from a file are what type — and what must you do with numbers?

Answers: 1. Truncates it to empty immediately. 2. `with` auto-closes the file even if the code crashes. 3. A `dict` keyed by the header row. 4. Strings; convert with `int()` / `float()`.

Session 9 — Regular Expressions

1. In regex, what does `.` match? How do you match a literal dot?
2. Why write regex patterns as raw strings `r"..."`?
3. Which function do you use to (a) check the *whole* string matches, and (b) replace matches?
4. What does `re.search` return when there's no match, and what must you do before `.group()`?

Answers: 1. Any character (except newline); use `\.` for a literal dot. 2. So backslashes aren't treated as Python string escapes. 3. (a) `re.fullmatch`, (b) `re.sub`. 4. It returns `None`; check `if m:` before calling `m.group()` or you'll hit an `AttributeError`.

Session 10 — Modules, OOP & Pythonic

1. In a class, what is `self`?
2. What happens if you iterate a generator twice?
3. Why doesn't a module's `if __name__ == "__main__":` block run when you `import` it?
4. What does a `@property` setter let you do that a plain attribute can't?

Answers: 1. The current instance ("this particular object"). 2. The second pass is empty — a generator is exhausted after one iteration. 3. On import, `__name__` is the module's name, not `"__main__"`, so the block is skipped. 4. Validate (or transform) the value on every assignment, so the object can reject bad data.

Scoring guide (formative, not graded)

- **All correct, explained why:** ready for the capstone.
- **Right answer, fuzzy why:** re-do that session's `traps-and-gotchas` rows.
- **Wrong on Session 2 items:** revisit Session 2 before continuing — it's load-bearing.